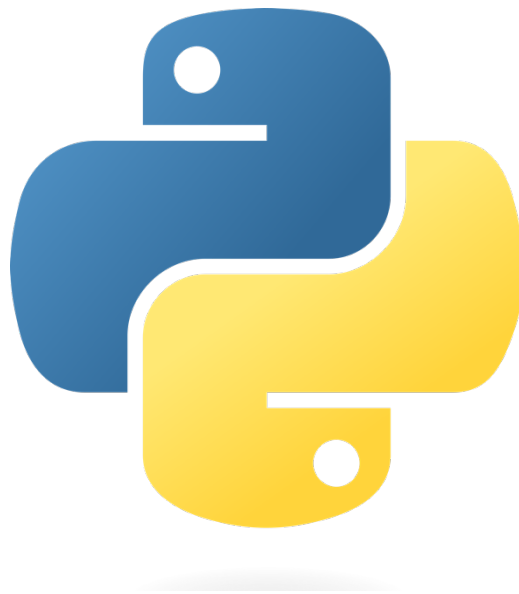


5. Exception Handling



PHAN DY

MIT, Build Bright University

Tell: 097 239 3060

Email: phandy010@gmail.com

1. Python Try Except

The **try** block lets you test a block of code for errors.

The **except** block lets you handle the error.

The **else** block lets you execute code when there is no error.

The **finally** block lets you execute code, regardless of the result of the try- and except blocks.

1.2. Exception Handling

When an error occurs, or exception as we call it, Python will normally stop and generate an error message.

These exceptions can be handled using the **try** statement:

Example 1

The **try** block will generate an exception, because **x** is not defined:

```
try:  
    print(x)  
except:  
    print("An exception occurred")
```

Since the try block raises an error, the except block will be executed.

Without the try block, the program will crash and raise an error:

Example 2

This statement will raise an error, because **x** is not defined:

```
print(x)
```

1.3. Many Exceptions

You can define as many exception blocks as you want, e.g. if you want to execute a special block of code for a special kind of error:

Example 1

Print one message if the try block raises a `NameError` and another for other errors:

```
try:
    print(x)
except NameError:
    print("Variable x is not defined")
except:
    print("Something else went wrong")
```

1.4. Else

You can use the `else` keyword to define a block of code to be executed if no errors were raised:

Example 1

In this example, the `try` block does not generate any error:

```
try:
    print("Hello")
except:
    print("Something went wrong")
else:
    print("Nothing went wrong")
```

1.5. Finally

The `finally` block, if specified, will be executed regardless if the try block raises an error or not.

Example 1

```
try:
    print(x)
except:
    print("Something went wrong")
finally:
    print("The 'try except' is finished")
```

This can be useful to close objects and clean up resources:

Example 2

Try to open and write to a file that is not writable:

```
try:
    f = open("demofile.txt")
    try:
        f.write("Lorum Ipsum")
    except:
        print("Something went wrong when writing to the file")
    finally:
        f.close()
except:
    print("Something went wrong when opening the file")
```

The program can continue, without leaving the file object open.

1.6. Raise an exception

As a Python developer you can choose to throw an exception if a condition occurs.

To throw (or raise) an exception, use the `raise` keyword.

Example 1

Raise an error and stop the program if x is lower than 0:

```
x = -1

if x < 0:
    raise Exception("Sorry, no numbers below zero")
```

The `raise` keyword is used to raise an exception.

You can define what kind of error to raise, and the text to print to the user.

Example 2

Raise a `TypeError` if x is not an integer:

```
x = "hello"

if not type(x) is int:
    raise TypeError("Only integers are allowed")
```

Reference

1. https://www.w3schools.com/python/python_try_except.asp